Trending

Insight Into ESP32 Sleep Modes & Their Power Consumption

There is no question that ESP32 is a worthy competitor to many WiFi/MCU SoCs out there, often beating it on both performance and price. But, depending on which state it's in, the ESP32 can be a relatively **power-hungry** device.

When your IoT project is powered by a plug in the wall, you tend not to care too much about power consumption. But if you are going to power your project by batteries, every mA counts.

The solution here is to cut back ESP32's power usage by leveraging one of its **Sleep Modes**. It's really a great strategy for dramatically extending the battery life of a project that doesn't need to be active all the time.

What is ESP32 sleep mode?

ESP32 Sleep mode is a power-saving state that ESP32 can enter when not in use. The ESP32's state is maintained in RAM. When ESP32 enters sleep mode, power is cut to any unneeded digital peripherals, while RAM receives just enough power to enable it to retain its data.

Inside ESP32 chip

In order to understand how ESP32 achieves power saving, we need to know what's inside the chip. The following illustration shows function block diagram of ESP32 chip.

At the heart of the ESP32 chip is a Dual-Core 32-bit microprocessor along with 448 KB of ROM, 520 KB of SRAM and 4MB of Flash memory.

It also contains WiFi module, Bluetooth Module, Cryptographic Accelerator (a co-processor designed specifically to perform cryptographic operations), the RTC module, and lot of peripherals.

ESP32 Power Modes

Thanks to the ESP32's advanced power management, it offers **5 configurable power modes**. As per the power requirement, the chip can switch between different power modes. The modes are:

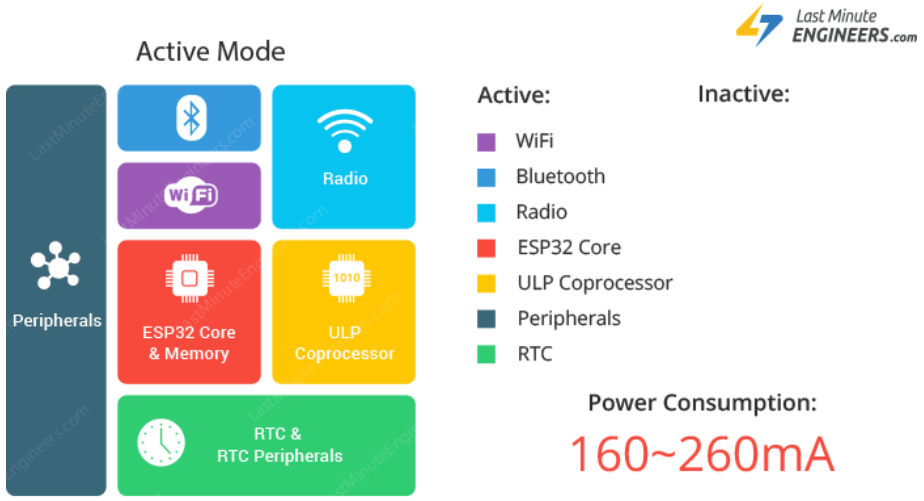
- Active Mode
- Modem Sleep Mode
- Light Sleep Mode
- Deep Sleep Mode
- Hibernation Mode

Each mode has its own distinct features and power saving capabilities. Let's look in to them one by one.

ESP32 Active Mode

The normal mode is also known as **Active Mode**. In this mode all the features of the chip are active.

As the active mode keeps everything (especially the WiFi module, the Processing Cores and the Bluetooth module) ON at all times, the chip requires more than 240mA current to operate. Also we observed that if you use both WiFi and Bluetooth functions together, sometimes high power spikes appear (biggest was 790mA).



If you look at the ESP32 datasheet, power consumption during Active power mode, with RF working is as follows:

Mode	Power Consumption
Wi-Fi Tx packet 13dBm~21dBm	**160~260mA**
Wi-Fi/BT Tx packet 0dBm	**120mA**
Wi-Fi/BT Rx and listening	**80~90mA**

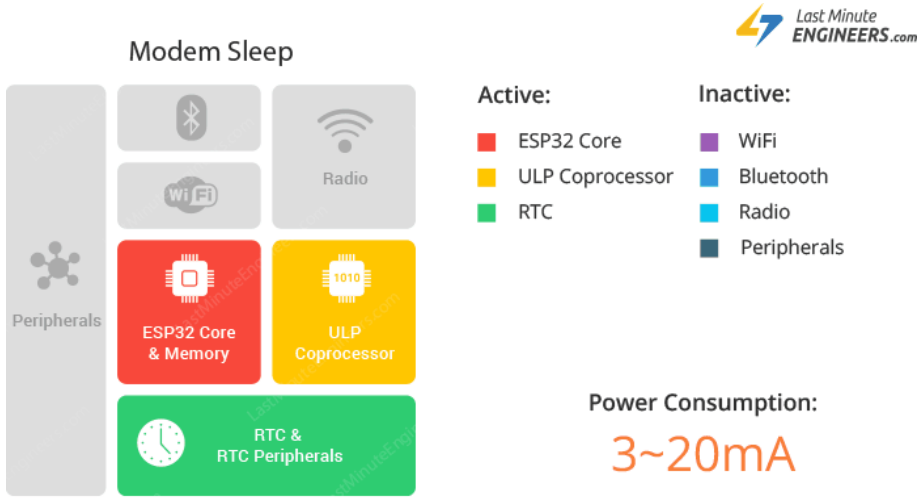
Power consumption during Active mode

Obviously, this is the most inefficient mode and will drain the most current. So, if we want to conserve power we have to disable them (by leveraging one of the other power modes) when not in use.

ESP32 Modem Sleep

In modem sleep mode everything is active while only WiFi, Bluetooth and radio are disabled. The CPU is also operational and the clock is configurable.

In this mode the chip consumes around 3mA at slow speed and 20mA at high speed.



To keep WiFi/Bluetooth connections alive, the CPU, Wi-Fi, Bluetooth, and radio are woken up at predefined intervals. It is known as **Association sleep pattern**.

During this sleep pattern, the power mode switches between the active mode and Modem sleep mode.

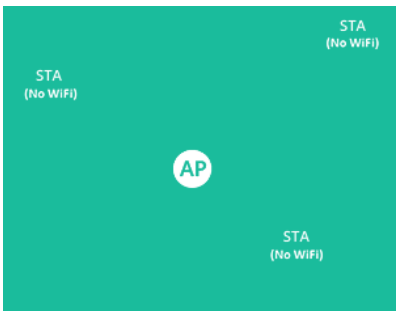
ESP32 can enter modem sleep mode only when it connects to the router in station mode. ESP32 stays connected to the router through the **DTIM beacon mechanism**.

In order to save power, ESP32 disables the Wi-Fi module between two DTIM Beacon intervals and wakes up automatically before the next Beacon arrival.

The sleep time is decided by the DTIM Beacon interval time of the router which is usually 100ms to 1000ms.

What is DTIM beacon mechanism?

DTIM is acronym for Delivery Traffic Indication Message.



In this mechanism, the access point(AP)/router transmits beacon frames periodically. Each frame contains all the information about the network. It is used to announce the presence of a wireless network and synchronize all the connected members.

ESP32 Light Sleep

The working mode of light sleep is similar to that of modem sleep. The chip also follows association sleep pattern.

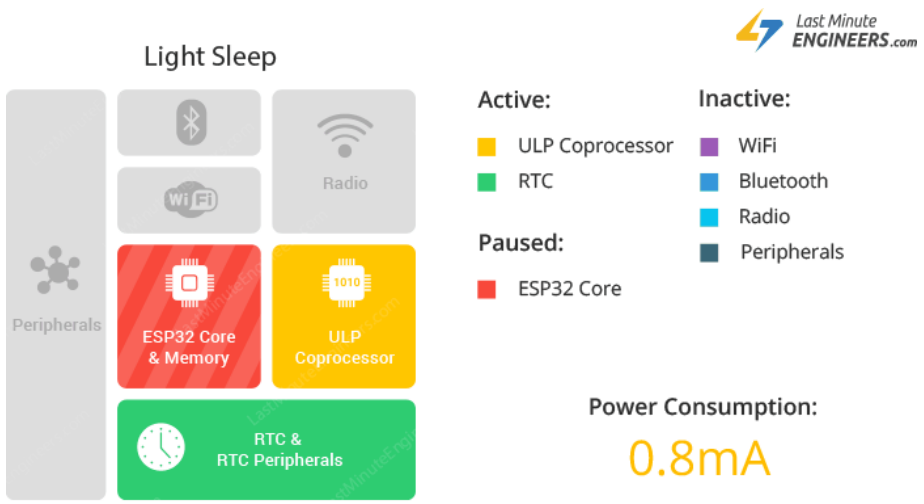
The difference is that, during light sleep mode, digital peripherals, most of the RAM and CPU are clock-gated.

What is Clock Gating?

Clock gating is a technique for reducing the dynamic power consumption.

It disables portions of the circuitry by powering off clock pulses, so that the flip-flops in them do not have to switch states. As switching states consumes power, when not being switched, the power consumption goes to zero.

During light sleep mode, the CPU is paused by powering off its clock pulses, while RTC and ULP-coprocessor are kept active. This results in less power consumption than in modem sleep mode which is around 0.8mA.



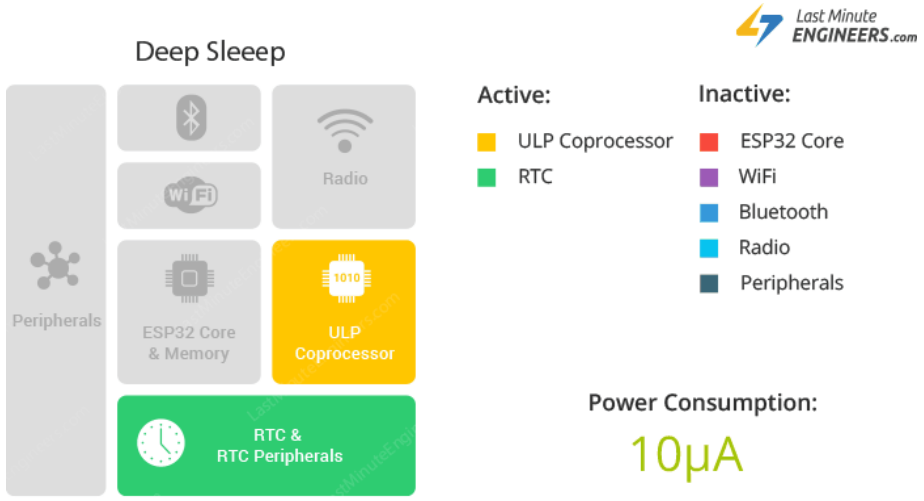
Before entering light sleep mode, ESP32 preserves its internal state and resumes operation upon exit from the sleep. It is known **Full RAM Retention**.

`esp_light_sleep_start()` function can be used to enter light sleep once wake-up sources are configured.

ESP32 Deep Sleep

In deep sleep mode, the CPU, most of the RAM and all the digital peripherals are powered off. The only parts of the chip that remains powered on are: RTC controller, RTC peripherals (including ULP co-processor), and RTC memories (slow and fast).

The chip consumes around 0.15 mA(if ULP co-processor is powered on) to 10μA.



During deep sleep mode, the main CPU is powered down, while the ULP co-processor does sensor measurements and wakes up the main system, based on the measured data from sensors. This sleep pattern is known as **ULP sensor-monitored pattern**.

Along with the CPU, the main memory of the chip is also disabled. So, everything stored in that memory is wiped out and cannot be accessed.

However, the RTC memory is kept powered on. So, its contents are preserved during deep sleep and can be retrieved after we wake the chip up. That's the reason, the chip stores Wi-Fi and Bluetooth connection data in RTC memory before disabling them.

So, if you want to use the data over reboot, store it into the RTC memory by defining a global variable with `RTC_DATA_ATTR` attribute. For example, `RTC_DATA_ATTR int bootCount = 0;`

In Deep sleep mode, power is shut off to the entire chip except RTC module. So, any data that is not in the RTC recovery memory is lost, and the chip will thus restart with a reset. This means program execution starts from the beginning once again.

TIP

ESP32 supports running a deep sleep wake stub when coming out of deep sleep. This function runs immediately as soon as the chip wakes up – before any normal initialization, bootloader code has run. After the wake stub runs, the chip can go back to sleep or continue to start normally.

Unlike the other sleep modes, the system cannot go into Deep-sleep mode automatically. `esp_deep_sleep_start()` function can be used to immediately enter deep sleep once wake-up sources are configured.

By default, ESP32 will automatically power down the peripherals not needed by the wake-up source. But you can optionally decide what all peripherals to shut down/keep on. For more information, check out API docs.

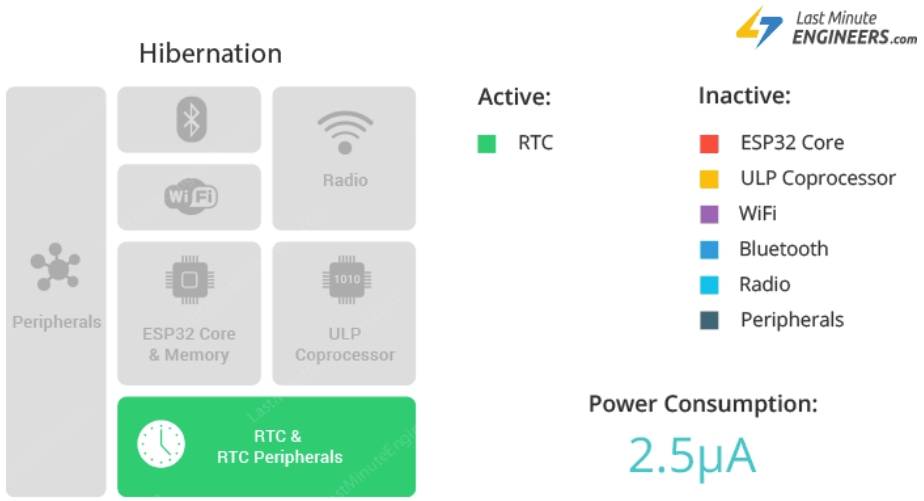
To know more about ESP32 Deep Sleep & its wake-up sources, please visit below tutorial.

ESP32 Deep Sleep & Its Wake-up Sources

Have you ever wanted your IoT project to last on batteries for almost 5 years? Wait... What? 5 years? Yes. It might sound ridiculous,...

ESP32 Hibernation mode

Unlike deep sleep mode, in hibernation mode the chip disables internal 8MHz oscillator and ULP-coprocessor as well. The RTC recovery memory is also powered down, meaning there's no way we can preserve any data during hibernation mode.

Everything else is shut off except only one RTC timer on the slow clock and some RTC GPIOs are active. They are responsible for waking up the chip from the hibernation mode.

This reduces power consumption even further. The chip consumes around 2.5μA only in hibernation mode.